

CS 4910 Lab 1 Report

Secret-Key Encryption Lab

Notes: (IMPORTANT)

- It is required to use this report template.
- Report your work in each section. **Describe** what you have done and what you have observed. You should take screenshots to support your description. You also need to provide an explanation of the observations that are interesting or surprising. Please also list the important code snippets followed by an explanation.
- Simply attaching code or screenshots without any explanation will NOT receive credits.
- **Do NOT claim anything you didn't do.** If you didn't try on a certain task, leave that section blank. You will receive a ZERO for the whole assignment if we find any overclaim.
- Grading will be based on your **description** and the completion of each task.
- After you finish, export this report as a PDF file and submit it on Canvas.

Your Full Name: Caroline Marie Duncan

Email Address: cduncan2@uccs.edu

I, Caroline Duncan, have read and understood the course academic integrity policy.

(Your report will not be graded without filling in the above AI statement)

1. Task 1: Frequency Analysis

Task 1 involved decrypting a monoalphabetic substitution using frequency analysis. This was done by analyzing how often letters/letter sequences appear in the ciphertext and comparing them to English frequency patterns.

The freq.py script was executed on ciphertext.txt that was provided to obtain the frequency distributions of 1-grams, 2-grams, and 3-grams.

The top results from the frequency analysis were:

1-gram (top): n: 488, y: 373, v: 348, x: 291, u: 280, q: 276

2-gram (top): yt: 115, tn: 89, mu: 74, nh: 58, vh: 57

3-gram (top): ytn: 78, vup: 30, mur: 20, ynh: 18

```

seed@seed:~/Downloads/labsetop-err/files$ tr 'ytn' 'THE' < ciphertext.txt | head -30
THE xqavhq Tzhu xu qzupvd lHmaH qEEcQ vgxzT hmrHT vbTEh THmq lxur qThvurE
vlvhpq Thme THE gvrrEh bEElQ lmsE v uxuvrEuvhmvu Txx

THE vlvhpq hvaE lvq gxxSEupEp gd THE pECnqE xb HvHFED lEmuqTEmu vT ntQ xzTqET
vup THE veevHEuT ncelxqnxu xb Hmq bmtc axcevud vT THE Eup vup mT lvq qHveEp gd
THE EcEhrEuaE xb cETxx TmcEq ze glvasrXlu exlnTnaq vhaavupd vaTmfncQ vup
v uvTmxuvl axufEHqVTmxu vq ghNEb vup cvp vq v bEFeh pHEvc vgxzT LHETHEh THEhE
xzrHT Tx gE v eHEqmpEUT lnuBHED THE qEVqxu pmpuT ozqT qEEc EkThv lxur mT lvq
EkThv lxur qEavzQE THE xqavhq lEHc cXfEP Tx THE bnhqT lEEsEup mu cvhah Tx
vfxmp axubinaTmur lATH THE atxqmur aHEcXud xb THE lnuTEh xldcenaq THvusq
edExuraHvur

xuE gnr jzEqTnxu qzhxzupmur THmq dEvhq vavpEcd vlvhpq mq Hxl xh mb THE
aHEcXud lml vpphEqQ cETxx EgeEanvild vbTEh THE rxlpEu rlxgEq lHmaH gEavcE
v ozgmIvuT axcmurxzT evHTd bxh TmcEq ze THE cxFEcEuT qeEvHHEvpEp gd
exlEhbzl Hxldlxp lxcEu lHx HEleEp hvnqE cmlnxuq xb pxliVhq Tx bmrHT qEkzvl
HvhvqqcEuT vhxzup THE axzuthd

qmruvlur THENh qzeexhT rxlpEu rlxgEq VTTEupEEq qlvTHEp THEcQElfEq mu glvas
qexhTEp lveEl emuq vup qxzupEp xbb vgxzT qEknqT exlEH mcgvlvuaEq bhxc THE hEP
avheET vup THE qTVrE xu THE vnh E lvq avliEp xzT vgxzT evd muEjzmTd vbTEh
mTq bxhcEh vuaHxh avTT qvplEH jzmT xuaE qHE lEVhuEP THvT qHE lvq cvsmur bvh
lEEq THvu v cvlE axhxqT vup pihmur THE aHEcXud uvTVlNE exhtcvu Txxs v glzuT
vup qvTmqbDmur pnr vT THE vliCVlE hxqTEh xb uxcmuVTEp pmhEaTxhq Hxl axzip
THvT gE TxeeEp

vq mT Tzhuq xzT vT lEvqT mu TEhcq xb THE xqavhq mT ehxgvglD lxuT gE

lxcEu mufxlfEp mu TmcEq ze qvmp THvT vLTHxzh THE rlxgEq qmrumbEp THE
munTmvTmfEq lvzuaH THED uEFeh muTEupEp mT Tx gE ozqT vu vlvhpq qEVqxu

```

Figure 1: 1-gram, 2-gram, and 3-gram frequency distributions

The frequency data was compared against known English letter frequencies. The top trigram 'ytn' (78 times) definitely corresponds to 'THE'. This gives: 'y' = 'T', 't' = 'H', 'n' = 'E'. This was confirmed by the top 2-gram 'yt' (115 times) = 'TH' and 'tn' (89 times) = 'HE'. 'n' is also the most frequent single letter (488 times), which is consistent with E being the most common letter in English. 'mu' (74 times) with 'n' = 'E' close to it suggests 'mu' = 'IN', giving 'm' = 'I' and 'u' = 'N'. 'nh' (58 times) = 'ER' gives 'h' = 'R'.

The first decryption attempt was run using this command:

```
tr 'ytn' 'THE' < ciphertext.txt | head -30
```

After applying the command, the partially decoded text was examined to identify more letter patterns. After analyzing the most frequent 2- and 3-letter cipher words, here is the new findings:

- 'xb' (17 times) - 'OF': x = O, b = F
- 'yx' (11 times) - 'TO': y = T, x = O
- 'vup' (25 times) - 'AND': v = A, u = N, p = D
- 'mur:' ING gives 'r' = G

tr commands were used to apply growing sets of substitutions and read the partially-decoded text:

```
tr 'ytnmuhxbvzepiqlgrcdf' 'THERINOFASYBNDWLGCPU' < ciphertext.txt | head -40
```

```

$ cd /Downloads/LabSetup-arn/Files$ tr 'ytnvmuhpr' 'THEORINGS' < ciphertext.txt | head -5
THE xqaONq TzNI xI qzIGod LhRAH qEEcQ OgxzT NRSHT OBTEH THRq iXIS qTNOISE
OLONGq TNRe THE gOSSEN bEEIq lRsE O iXIOSEIONROI Txx

THE OLONGq NOaE loq gxxsEIEG gd THE GECRqE xb HONFED lERiQTERI OT RTq xzTqET
OIG THE OeeONEIT RcelxqRxi xb HRq bRlc axceOId OT THE EIG OIG RT loq qHOeEG gd
THE ECENSEIaE xb cETxx TRceq ze gIoassxli exlRTRaq ONcaOIGd OaTRFRqc OIG
O IOTRxiOl axifENqOTRxi Oq gNREb OIG cOG Oq O bEFEN GNEOC OgxzT LHETHEN THENE
xzSHT Tx gE O eNEqRGEIT lRiBNED THE qEOqxi GRGIT ozqT qEEc EKTNO lXIS RT loq
EKTNO lXIS gEaOzqE THE xqaONq lENE cxFEg TX THE BRNqT lEESEIG RI cONaH TX
DFVRG axiBIRaTRiS lRTH THE aLxqRiS aENEcxid xb THE lRITEN xldceRaQ THOISq
edExISaHOIS

XIE gRS jzEqTRxi qzNNxzIGRIS THRq dEONq OaOGEd OLONGq Rq HxI xN Rb THE
aENEcxid lRli OCGNEqq cETxx EgeEaROlId OBTEH THE SxIgeI SlxgEq LhRAH gEaOce
O ozgRlOIT axCRISxzT eONTD bXN TRceq ze THE cxFEcEIT qeEONHEOGEG gd
exlENbzl HxldlxxG lxcEI lHx HeteEG NORqE cRlIRxiQ xb GxllONq Tx BRsHT qEkzOl
HONqgcEIT ONxzIG THE axzITNd

GRSIOIRIS THERN qzeexNT SxIgeI SlxgEq OTTEIGEEq qLOTHEG THEcqElFEq RI gIoas
qexNTEG lOeI eRIq OIG qxzIGEG xbb OgxzT qEkRqT exlEN RcgOlOiaeq bNxc THE NEG
aONeET OIG THE qTOSE xI THE ORN E loq aOlEG xzT OgxzT eOd RIEjzRTd OBTEH
RTq bXNCEN OIAHXN aOTT qOGLen jzRT xIaE qHE lEONIEG THOT qHE loq cOSRIS BON
IEqq THOI O cOlE axHXqT OIG GzNRIS THE aENEcxid IOTOLRE exNTCOI Txxs O gIzIT
OIG qOTRqbdRIS GRS OT THE OlIcOlE NqXTEN xb lxcRIOTEG GRNEaTxNq HxI axzIG
THOT gE TXeeEG

Dq RT TzNIq xzT OT lEOqT RI TENcQ xb THE xqaONq RT eNxgOgId lXIT gE

lxcEI RIfxIfeG RI TRceq ze qORg THOT OlTHxzSH THE SlxgEq qRSIRbREG THE
RIRTROTRfeq lOzIaH THEd IEFEN RITEIGEG RT Tx gE ozqT OI OLONGq qEOqXI
aOceORSI xN xIE THOT gEaOce OgqxaROTEG xId lRTH NEGaONeET OaTRxiQ RIqTEOG
O qexsEqLxcOI qORg THE SNxze Rq lXNsRIS gEHRIg aLxqEG GxxNq OIG HOq qRIaE
O cOqQEG cRlIRxi bXN RTq lESOl GEbEIQE bZIG LhRAH OBTEH THE SlxgEq loq
bLxxGEG lRTH THxzqOIGq xb GxIOTRxiQ xb xN lEqq bNxc eXexIE RI qxcE
axzITNREG

IX aOl Tx lEON gIoas SxliQ lEIT xzT RI OGfOiaE xb THE xqaONq THxzSH THE
cxFEcEIT lRli OlcxqT aENTORId gE NEbENEIaEG gEbXNE OIG GzNRIS THE aENEcxid
EgeEaROlId qRIaE fxaOl cETxx qzeexNTENq lRsE OqHlEd ozGG lOzNO GENI OIG

```

Figure 2: Partial decryption after applying initial letter mappings

```

$ cd /Downloads/LabSetup-arn/Files$ tr 'ytnvmuhprotglbzxqaecdf' 'THEORINGSADNBUFLATCYP' < ciphertext.txt | head -50
THE FLAONL TUNI FI LUIGOY WHRAH LEECL ONFUT NRSHT OBTEH THRL DFIS LTNOISE
OWONGL TNRT THE NOSSEN BEEDL DRSE O IFIOSEIONROI TFF

THE OWONGL NOAE WOL NFFSEIEG NY THE GECRL FB HONPEY WERILTERI OT RTL FUTLET
OIG THE OTTONEIT RCTDFLRFI FB HRL BRDC AFCTOIY OT THE EIG OIG RT WOL LHOTEG NY
THE ECENSEIAE FB cETFF TRCEL UT NROASSRNI TFDRTAL ONCAOIOY OATRRPLC OIG
O IOTRfIOO AFIPENLOTRFI Ol NNRREb OIG cOG Ol O bEFEN GNEOC ONFUT WHETHEH THENE
FUSHT TF NE O lNELRGIT WRIBNEY THE LEOLFI GRGIT AUlT lEEc EKTNO DFIS RT WOL
EKTNO DFIS NEaOULE THE FLAONL WENE CFPEG TF THE BRNLT WEESEIG RI cONaH TF
OPFRG AFIBDRATRIS WRTH THE ADfLRIS aENEcfiy FB THE WRITEN FDYCTRAL THOISL
TYEFISaHOIS

FIE NRS jUELTRFI LUNNFUIGRIS THRL YEONL OAOGECY OWONGL RL HFW FN RB THE
aENEcfiy WRDD OGNELL CETFF ELTEARODDY OBTEH THE SFDGEI SDFNEL WHRAH NEAOCE
O AUNRDOIT AFCRISFUT TONTY BFN TRCEL UT THE CFPECEIT lTEONHEOGEG NY
TFWENBUD HFDdYwFFG WFCEI WHF HEDTEG NORLE CRDDRfIL FB GFDDOONL TF BRsHT LEKUOD
HONOLLCEIT ONFUIG THE AFUITNY

LRSIODRIS THERN LUTTFNT SFDGEI SDFNEL OTTEIGEEl LWOTHEG THEcLDEPLe RI NDOAS
LTFNTEG DOTE TRIL OIG LFUIGEG FBB ONFUT LEKRLT TFWEN RCNODOIAEL BNFC THE NEG
aONTET OIG THE lTOSE FI THE ORN E WOL ADDEG FUT ONFUT TOY RIEJURTY OBTEH
RTL BFNcEN OIAHFN aOTT LOGDEN JURT FIAE LHE DEONIEG THOT LHE WOL cOSRIS BON
DELL THOI O CODE AFHFLT OIG GUNRIS THE aENEcfiy IOTODRE TFNTCOI TFFS O NDUIT
OIG LOTRLBYRIS GRS OT THE ODDCODE NFLTEN FB lFCRIOTEG GRNEaTFNL HFW AFUDG
THOT NE TFFTEG

OL RT TUNIL FUT OT DEOLT RI TENCL FB THE FLAONL RT TNFNONDY WFIT NE

WFCEI RlPFODEG RI TRCEL UT LOrg THOT ODFHUSH THE SDFNEL LRSIRbREG THE
RIRTROTRPEL DOUIAH THEY lEPEN RITEIGEG RT TF NE AUlT OI OWONGL LEOLFI
aOCTORSI FN FIE THOT NEAOCE OLLfAROTEG FIDY WRTH NEGaONTET OATRFIL RILTEOG
O lTfSELWfCOI LOrg THE SNFUT RL WfNsRIS NEHRIG ADfLEG GFfNL OIG HOL LRiAE
O COLLEG CRDDRfI BFN RTL DESOD GEbEILE BUTG WHRAH OBTEH THE SDFNEL WOL
BDFfGEG WRTH THFULOIGL FB GFIOTRfIL FB FN DELL BNFC TEFTDE RI LFCE
AFUITNREL

IF ADD TF WEON NDOAS SFWIL WEIT FUT RI OGPOIAE FB THE FLAONL THFUSH THE
CFPECEIT WRDD ODCFLT aENTORIDY NE NEbENEIaEG NEbNE OIG GUNRIS THE aENEcfiy
ELTEARODDY LRiAE PFAOD CETFF LUTTFNTEN DRSE OlHDEY AUGG DOUNO GENI OIG
lRAFDE sRGCOI ONE LAHEGUEG TNElEITENL

DlETHEN REOTUNE FB THRI lEOLET TF ETf NFODDY sTEWl WHF RI sERIS TF WRT NEIT

```

Figure 3: Further decryption with expanded substitution set

As more letters were recovered, recognizable words began to appear in the somewhat decoded text. The words “OSCARS, HARVEY, and WEINSTEIN” became noticeable, eventually revealing that the plaintext is a news article about the Oscars ceremony, the Harvey Weinstein scandal and the MeToo movement.

Key deductions:

- 'xqavhq' = 'OSCARS'
- 'tvhfnd' = 'HARVEY'
- 'lnmuqynmu' = 'WEINSTEIN'
- 'qzupvd' = 'SUNDAY'
- 'vgxzy' = 'ABOUT'
- 'gxxsnupnp' = 'BOOKENDED'

```

seed: /dev/urandom | fold -w 64 | tr -dc 'abcdefghijklmnopqrstuvwxyz' | fold -w 64 | xxd -p | head -n 1
THE OSCARS TURN ON SUNDAY WHICH SEEMS ABOUT RIGHT AFTER THIS LONG STRANGE
AWARDS TRIP THE BAGGER FEELS LIKE A NONAGENARIAN TOO

THE AWARDS RACE WAS BOOKENDED BY THE DEMISE OF HARVEY WEINSTEIN AT ITS OUTSET
AND THE APPARENT IMPLOSION OF HIS FILM COMPANY AT THE END AND IT WAS SHAPED BY
THE EMERGENCE OF METOO TIMES UP BLACKGOWN POLITICS ARKANDY ACTIVISM AND
A NATIONAL CONVERSATION AS BRIEF AND MAD AS A FEVER DREAM ABOUT WHETHER THERE
OUGHT TO BE A PRESIDENT WINFREY THE SEASON DIDNT JUST SEEM EQTRA LONG IT WAS
EQTRA LONG BECAUSE THE OSCARS WERE MOVED TO THE FIRST WEEKEND IN MARCH TO
AVOID CONFLICTING WITH THE CLOSING CEREMONY OF THE WINTER OLYMPICS THANKS
PYEONGCHANG

ONE BIG QUESTION SURROUNDING THIS YEARS ACADEMY AWARDS IS HOW OR IF THE
CEREMONY WILL ADDRESS METOO ESPECIALLY AFTER THE GOLDEN GLOBES WHICH BECAME
A XUBILANT COMINGOUT PARTY FOR TIMES UP THE MOVEMENT SPEARHEADED BY
POWERFUL HOLLYWOOD WOMEN WHO HELPED RAISE MILLIONS OF DOLLARS TO FIGHT SEQUAL
HARASSMENT AROUND THE COUNTRY

SIGNALING THEIR SUPPORT GOLDEN GLOBES ATTENDEES SMASHED THEMSELVES IN BLACK
SPORTED LAPEL PINS AND SOUNDED OFF ABOUT SEXIST POWER IMBALANCES FROM THE RED
CARPET AND THE STAGE ON THE AIR E WAS CALLED OUT ABOUT PAY INEQUITY AFTER
ITS FORMER ANCHOR CATT SADLER JOINT ONCE SHE LEARNED THAT SHE WAS MAKING FAR
LESS THAN A MALE COHOST AND DYING THE CEREMONY HATILIE PORTMAN TOOK A BLYHT
AND SATISFYING DIG AT THE ALLMALE ROSTER OF NOMINATED DIRECTORS HOW COULD
THAT BE TOPPED

AS IT TURNS OUT AT LEAST IN TERMS OF THE OSCARS IT PROBABLY WOHT BE
WOMEN INVOLVED IN TIMES UP SAID THAT ALTHOUGH THE GLOBES SIGNIFIED THE
INITIATIVES LAUNCH THEY NEVER INTENDED IT TO BE XYST AN AWARDS SEASON
CAMPAIGN OR ONE THAT BECAME ASSOCIATED ONLY WITH REDCARPET ACTIONS INSTEAD
A SPOKESWOMAN SAID THE GROUP IS WORKING BEHIND CLOSED DOORS AND HAS SINCE
AMASSED MILLION FOR ITS LEGAL DEFENSE FYND WHICH AFTER THE GLOBES WAS
FLOODED WITH THOUSANDS OF DONATIONS OF OR LESS FROM PEOPLE IN SOME
COUNTRIES

HO CALL TO WEAR BLACK GOWNS WENT OUT IN ADVANCE OF THE OSCARS THOUGH THE
MOVEMENT WILL ALMOST CERTAINLY BE REFERENCED BEFORE AND DYING THE CEREMONY
ESPECIALLY SINCE VOCAL METOO SYPPORTERS LIKE ASHLEY XYDD LAYRA DERH AND
HICOLE KIDMAH ARE SCHEDVLED PRESENTERS

ANOTHER FEATYRE OF THIS SEASON HO ONE REALLY KNOWS WHO IS GOING TO WIN BEST
PICTYRE ARGVABLY THIS HAPPENS A LOT OF THE TIME INARGVABLY THE HAILBITER

```

Figure 4: Nearly complete decryption

After resolving all 26 letter mappings, the final decryption command was:

```
tr 'abcdefghijklmnopqrstuvwxyz' 'CFMYPVBRLJQWIEXDSGKHNAZOTU' <
ciphertext.txt
```

This decryption command produced the final substitution key: A maps to C, B to F, C to M, D to Y, E to P, F to V, G to B, H to R, I to L, J to J, K to Q, L to W, M to I, n to E, o to X, p to D, q to S, r to G, s to K, t to H, u to N, v to A, w to Z, x to O, y to T, and z to U.

```

seed: /dev/urandom | fold -w 64 | tr -dc 'abcdefghijklmnopqrstuvwxyz' | fold -w 64 | xxd -p | head -n 20
THE OSCARS TURN ON SUNDAY WHICH SEEMS ABOUT RIGHT AFTER THIS LONG STRANGE
AWARDS TRIP THE BAGGER FEELS LIKE A NONAGENARIAN TOO

THE AWARDS RACE WAS BOOKENDED BY THE DEMISE OF HARVEY WEINSTEIN AT ITS OUTSET
AND THE APPARENT IMPLOSION OF HIS FILM COMPANY AT THE END AND IT WAS SHAPED BY
THE EMERGENCE OF METOO TIMES UP BLACKGOWN POLITICS ARKANDY ACTIVISM AND
A NATIONAL CONVERSATION AS BRIEF AND MAD AS A FEVER DREAM ABOUT WHETHER THERE
OUGHT TO BE A PRESIDENT WINFREY THE SEASON DIDNT JUST SEEM EQTRA LONG IT WAS
EQTRA LONG BECAUSE THE OSCARS WERE MOVED TO THE FIRST WEEKEND IN MARCH TO
AVOID CONFLICTING WITH THE CLOSING CEREMONY OF THE WINTER OLYMPICS THANKS
PYEONGCHANG

ONE BIG QUESTION SURROUNDING THIS YEARS ACADEMY AWARDS IS HOW OR IF THE
CEREMONY WILL ADDRESS METOO ESPECIALLY AFTER THE GOLDEN GLOBES WHICH BECAME
A XUBILANT COMINGOUT PARTY FOR TIMES UP THE MOVEMENT SPEARHEADED BY
POWERFUL HOLLYWOOD WOMEN WHO HELPED RAISE MILLIONS OF DOLLARS TO FIGHT SEQUAL
HARASSMENT AROUND THE COUNTRY

SIGNALING THEIR SUPPORT GOLDEN GLOBES ATTENDEES SMASHED THEMSELVES IN BLACK
SPORTED LAPEL PINS AND SOUNDED OFF ABOUT SEXIST POWER IMBALANCES FROM THE RED

```

Figure 5: Sample of the fully decrypted plaintext

2. Task 2: Encryption using Different Ciphers and Modes

Task 2 involved using openssl with three different cipher types. A file called 'plain.txt' was created and the following command syntax was used:

```
openssl enc -<cipher> -e -in plain.txt -out cipher.bin -K <key> -iv <iv> -p
```

A test plaintext file was created using:

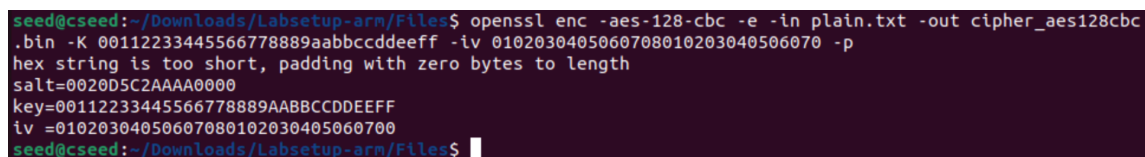
```
echo -n "This is a test message for encryption lab task 2." > plain.txt
```

The -n flag prevents echo from appending a newline character, keeping the file exactly 49 bytes.

Cipher 1:

```
openssl enc -aes-128-cbc -e -in plain.txt -out cipher_aes128cbc.bin -K
00112233445566778889aabbccddeeff -iv 01020304050607080102030405060708 -p
```

AES-128-CBC uses a 128-bit key. In CBC mode, each plaintext block is XORed with the previous ciphertext block before encryption, meaning the IV is needed to encrypt the first block. Padding is applied when the plaintext is not a multiple of 16 bytes so the final output will be 64 bytes.



```
seed@cseed:~/Downloads/Labsetup-arm/Files$ openssl enc -aes-128-cbc -e -in plain.txt -out cipher_aes128cbc
.bin -K 00112233445566778889aabbccddeeff -iv 01020304050607080102030405060708 -p
hex string is too short, padding with zero bytes to length
salt=0020D5C2AAAA0000
key=00112233445566778889AABBCCDDEEFF
iv =01020304050607080102030405060708
seed@cseed:~/Downloads/Labsetup-arm/Files$
```

Figure 6: AES-128-CBC

Cipher 2:

```
openssl enc -aes-256-cbc -e -in plain.txt -out cipher_aes256cbc.bin -K
00112233445566778889aabbccddeeff00112233445566778889aabbccddeeff -iv
01020304050607080102030405060708 -p
```

AES-256-CBC uses a 256 bit key, making it stronger than AES-128 but slower because there are more encryption rounds. The block size remains 16 bytes, so the same padding is added, the output is also 64 bytes. The ciphertext produced is different from AES-128-CBC despite the same plaintext and IV, because the larger key produces a different key schedule.

```
seed@cseed:~/Downloads/Labsetup-arm/Files$ openssl enc -aes-256-cbc -e -in plain.txt -out cipher_aes256cbc
.bin -K 00112233445566778889aabbccddeeff00112233445566778889aabbccddeeff -iv 01020304050607080102030405060
7080 -p
hex string is too long, ignoring excess
salt=002040B5AAAA0000
key=00112233445566778889AABBCCDDEEFF00112233445566778889AABBCCDDEEFF
iv =01020304050607080102030405060708
seed@cseed:~/Downloads/Labsetup-arm/Files$
```

Figure 7: AES-256-CBC

Cipher 3:

```
openssl enc -aes-128-cfb -e -in plain.txt -out cipher_aes128cfb.bin -K
00112233445566778889aabbccddeeff -iv 01020304050607080102030405060708 -p
```

AES-128-CFB operates in Cipher Feedback mode, which turns the block cipher into a stream cipher. CFB encrypts the IV to produce a keystream, which is then XORed with the plaintext. Since it operates as a stream cipher, no padding is required meaning the output is exactly the same size as the input.

```
seed@cseed:~/Downloads/Labsetup-arm/Files$ openssl enc -aes-128-cfb -e -in plain.txt -out cipher_aes128cfb
.bin -K 00112233445566778889aabbccddeeff -iv 01020304050607080102030405060708 -p
salt=002039C9AAAA0000
key=00112233445566778889AABBCCDDEEFF
iv =01020304050607080102030405060708
seed@cseed:~/Downloads/Labsetup-arm/Files$
```

Figure 8: AES-128-CFB

```
seed@cseed:~/Downloads/Labsetup-arm/Files$ ls -la plain.txt cipher_aes128cbc.bin cipher_aes256cbc.bin ciph
er_aes128cfb.bin
-rw-rw-r-- 1 seed seed 64 Feb 21 00:15 cipher_aes128cbc.bin
-rw-rw-r-- 1 seed seed 57 Feb 21 00:26 cipher_aes128cfb.bin
-rw-rw-r-- 1 seed seed 64 Feb 21 00:22 cipher_aes256cbc.bin
-rw-rw-r-- 1 seed seed 57 Feb 21 00:09 plain.txt
seed@cseed:~/Downloads/Labsetup-arm/Files$
```

Figure 9: Final Output

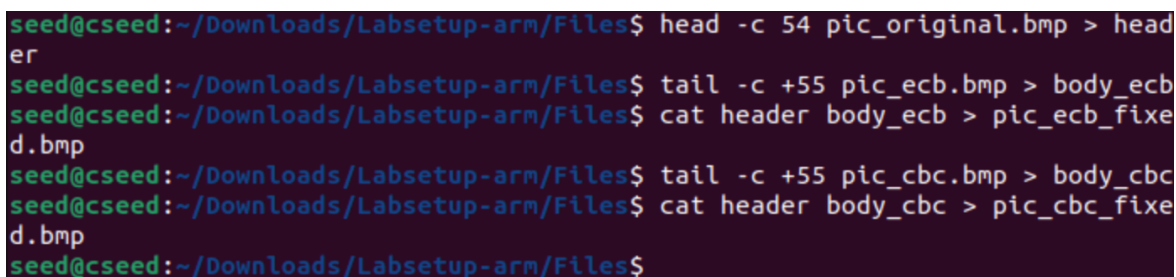
AES-128-CBC and AES-256-CBC both produce 64 byte outputs because CBC mode requires input to be padded to a multiple of the block size. Since CFB operates as a stream cipher and requires no padding, AES-128-CFB produces a 49 byte output matching the plaintext. The main difference between AES-128 and AES-256 is the key length. The 256 bit variant has a larger key space and more encryption rounds. All three ciphers require an IV, so that the same plaintext encrypted twice with the same key produces different ciphertext.

3. Task 3: Encryption Mode – ECB vs. CBC

Task 3 involves showing the security difference between ECB and CBC encryption modes by encrypting BMP image files and comparing the results. The original file (pic_original.bmp) was used as the first test image.

The BMP header (first 54 bytes) contains metadata that must be preserved for image viewers to recognize the file format. After encrypting, the following commands were used:

```
head -c 54 pic_original.bmp > header
tail -c +55 pic_ecb.bmp > body_ecb
cat header body_ecb > pic_ecb_fixed.bmp
tail -c +55 pic_cbc.bmp > body_cbc
cat header body_cbc > pic_cbc_fixed.bmp
```



```
seed@cseed:~/Downloads/Labsetup-arm/Files$ head -c 54 pic_original.bmp > header
seed@cseed:~/Downloads/Labsetup-arm/Files$ tail -c +55 pic_ecb.bmp > body_ecb
seed@cseed:~/Downloads/Labsetup-arm/Files$ cat header body_ecb > pic_ecb_fixed.bmp
seed@cseed:~/Downloads/Labsetup-arm/Files$ tail -c +55 pic_cbc.bmp > body_cbc
seed@cseed:~/Downloads/Labsetup-arm/Files$ cat header body_cbc > pic_cbc_fixed.bmp
seed@cseed:~/Downloads/Labsetup-arm/Files$
```

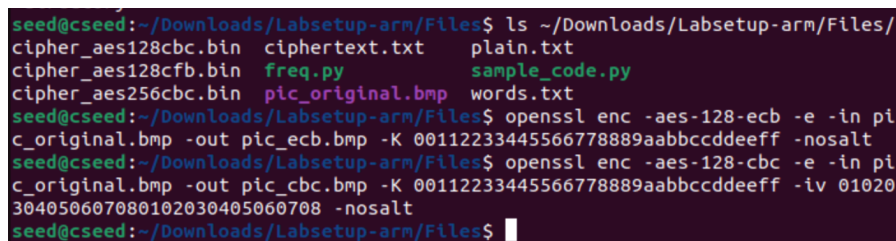
Figure 10 : Restoring Commands

Encryption commands:

```
openssl enc -aes-128-ecb -e -in pic_original.bmp -out pic_ecb.bmp -K
00112233445566778889aabbccddeeff -nosalt

openssl enc -aes-128-cbc -e -in pic_original.bmp -out pic_cbc.bmp -K
00112233445566778889aabbccddeeff -iv 01020304050607080102030405060708 -nosalt
```

The first image shows a red oval and a teal rectangle. When encrypted with ECB mode, the shapes are still clearly visible, the oval and rectangle outlines are visible the full structure of the image is exposed even though exact pixel values are obscured. The image encrypted with CBC is completely random with no shapes or outlines that the original image showed.



```
seed@cseed:~/Downloads/Labsetup-arm/Files$ ls ~/Downloads/Labsetup-arm/Files/
cipher_aes128cbc.bin  ciphertext.txt  plain.txt
cipher_aes128cfb.bin  freq.py        sample_code.py
cipher_aes256cbc.bin  pic_original.bmp words.txt
seed@cseed:~/Downloads/Labsetup-arm/Files$ openssl enc -aes-128-ecb -e -in pic_original.bmp -out pic_ecb.bmp -K 00112233445566778889aabbccddeeff -nosalt
seed@cseed:~/Downloads/Labsetup-arm/Files$ openssl enc -aes-128-cbc -e -in pic_original.bmp -out pic_cbc.bmp -K 00112233445566778889aabbccddeeff -iv 01020304050607080102030405060708 -nosalt
seed@cseed:~/Downloads/Labsetup-arm/Files$
```

Figure 11: Encryption Commands



Figure 14: Original File

```
seed@cseed:~/Downloads/Labsetup-arm/Files$ rm convert.py
seed@cseed:~/Downloads/Labsetup-arm/Files$ echo "from PIL import I
mage" > convert.py
seed@cseed:~/Downloads/Labsetup-arm/Files$ echo "img = Image.open(
'Cat_August_2010-4.jpeg')" >> convert.py
seed@cseed:~/Downloads/Labsetup-arm/Files$ echo "img.save('mycat.b
mp')" >> convert.py
seed@cseed:~/Downloads/Labsetup-arm/Files$ echo "print('done')" >>
convert.py
seed@cseed:~/Downloads/Labsetup-arm/Files$ python3 convert.py
done
seed@cseed:~/Downloads/Labsetup-arm/Files$
```

Figure 15: BMP Conversion

Encryption commands:

```
openssl enc -aes-128-ecb -e -in mycat.bmp -out mycat_ecb.bmp -K
00112233445566778889aabbccddeeff -nosalt

openssl enc -aes-128-cbc -e -in mycat.bmp -out mycat_cbc.bmp -K
00112233445566778889aabbccddeeff -iv 01020304050607080102030405060708 -nosalt
```

```

seed@cseed:~/Downloads/Labsetup-arm/Files$ python3 convert.py
done
seed@cseed:~/Downloads/Labsetup-arm/Files$ openssl enc -aes-128-ec
b -e -in mycat.bmp -out mycat_ecb.bmp -K 00112233445566778889aabbcc
ddee ff -nosalt
seed@cseed:~/Downloads/Labsetup-arm/Files$ openssl enc -aes-128-cb
c -e -in mycat.bmp -out mycat_cbc.bmp -K 00112233445566778889aabbcc
ddee ff -iv 01020304050607080102030405060708 -nosalt
seed@cseed:~/Downloads/Labsetup-arm/Files$ head -c 54 mycat.bmp >
header_mycat
seed@cseed:~/Downloads/Labsetup-arm/Files$ tail -c +55 mycat_ecb.b
mp > body_mycat_ecb
seed@cseed:~/Downloads/Labsetup-arm/Files$ cat header_mycat body_m
ycat_ecb > mycat_ecb_fixed.bmp
seed@cseed:~/Downloads/Labsetup-arm/Files$ tail -c +55 mycat_cbc.b
mp > body_mycat_cbc
seed@cseed:~/Downloads/Labsetup-arm/Files$ cat header_mycat body_m
ycat_cbc > mycat_cbc_fixed.bmp
seed@cseed:~/Downloads/Labsetup-arm/Files$

```

Figure 16: Encryption

Both of the ECB and CBC encrypted versions of the cat image are completely obscured, with no recognizable features visible in either output. After some research, this is because the image has complex textures, gradients, and varying pixel values. Since neighboring pixel blocks are rarely identical, ECB mode does not produce repeated ciphertext blocks and reveals no patterns. When the plaintext data has low repetition (or a very complex image) the vulnerability of ECB is not shown.

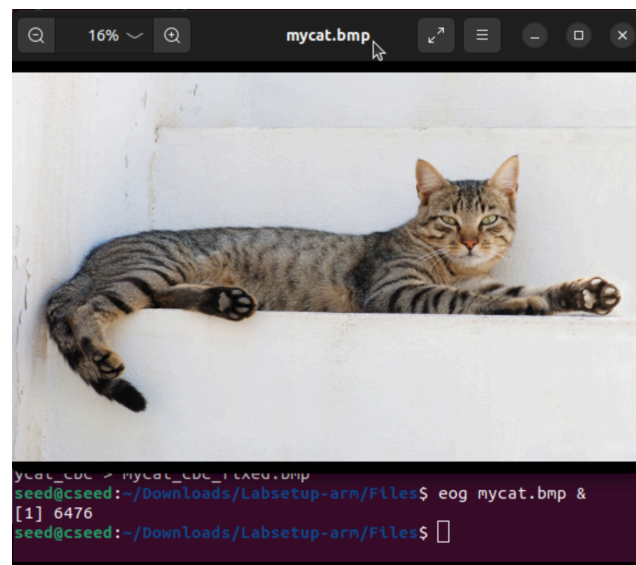


Figure 17: Original Picture

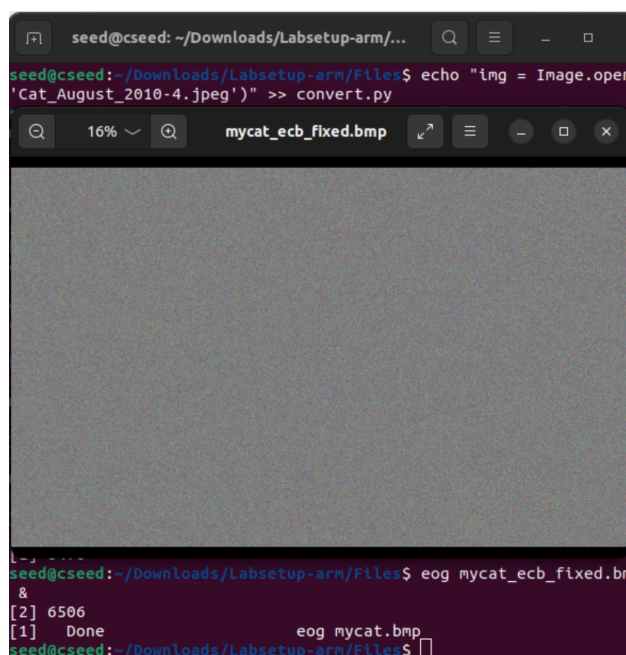


Figure 18: ECB Applied

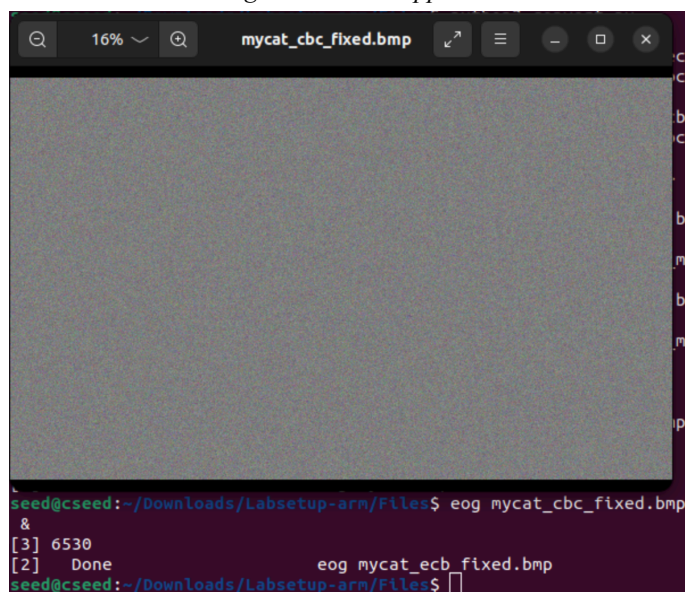


Figure 19: CBC Applied

ECB mode is insecure for structured or uniform data (simple images) because it encrypts each block independently and deterministically, meaning identical plaintext blocks always produce identical ciphertext blocks which is shown by the first image, where large uniform sections produce repeated ciphertext blocks that keep the original structure. CBC mode eliminates this by XORing each plaintext block with the previous ciphertext block, so that even identical plaintext blocks produce different ciphertext depending on their position. The second image shows that ECB's weakness is data-dependent. Complex images may not visually leak information but, this

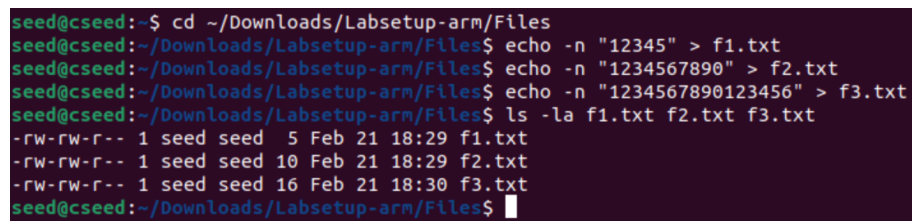
still does not make ECB secure, because it would still expose sensitive information in structured file formats, database records, or any data with repeating patterns.

4. Task 4: Padding

Task 4 involves showing how padding works with block ciphers. When the plaintext length is not a multiple of the block size, padding must be added to complete the final block. ECB and CBC require padding because they are block cipher modes and they process data in fixed 16 byte chunks and cannot handle partial blocks. CFB and OFB modes do not require padding because they act as stream ciphers.

Three test files were created with different lengths:

```
echo -n "12345" > f1.txt
echo -n "1234567890" > f2.txt
echo -n "1234567890123456" > f3.txt
```

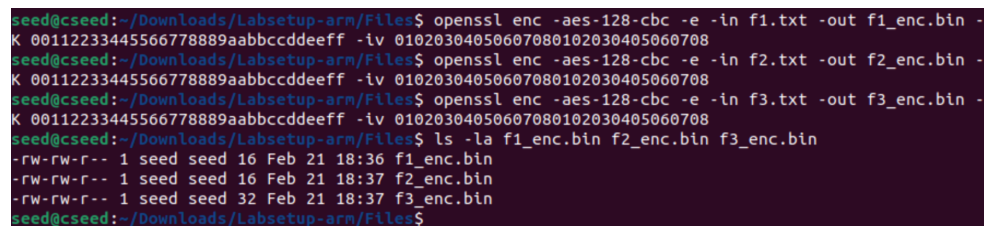


```
seed@cseed:~$ cd ~/Downloads/Labsetup-arm/Files
seed@cseed:~/Downloads/Labsetup-arm/Files$ echo -n "12345" > f1.txt
seed@cseed:~/Downloads/Labsetup-arm/Files$ echo -n "1234567890" > f2.txt
seed@cseed:~/Downloads/Labsetup-arm/Files$ echo -n "1234567890123456" > f3.txt
seed@cseed:~/Downloads/Labsetup-arm/Files$ ls -la f1.txt f2.txt f3.txt
-rw-rw-r-- 1 seed seed 5 Feb 21 18:29 f1.txt
-rw-rw-r-- 1 seed seed 10 Feb 21 18:29 f2.txt
-rw-rw-r-- 1 seed seed 16 Feb 21 18:30 f3.txt
seed@cseed:~/Downloads/Labsetup-arm/Files$
```

Figure 20: Creating Files

All three were encrypted using AES-128-CBC:

```
openssl enc -aes-128-cbc -e -in f1.txt -out f1_enc.bin -K 00112233445566778889aabbccddeeff
-iV 01020304050607080102030405060708
openssl enc -aes-128-cbc -e -in f2.txt -out f2_enc.bin -K 00112233445566778889aabbccddeeff
-iV 01020304050607080102030405060708
openssl enc -aes-128-cbc -e -in f3.txt -out f3_enc.bin -K 00112233445566778889aabbccddeeff
-iV 01020304050607080102030405060708
```



```
seed@cseed:~/Downloads/Labsetup-arm/Files$ openssl enc -aes-128-cbc -e -in f1.txt -out f1_enc.bin -K 00112233445566778889aabbccddeeff -iv 01020304050607080102030405060708
seed@cseed:~/Downloads/Labsetup-arm/Files$ openssl enc -aes-128-cbc -e -in f2.txt -out f2_enc.bin -K 00112233445566778889aabbccddeeff -iv 01020304050607080102030405060708
seed@cseed:~/Downloads/Labsetup-arm/Files$ openssl enc -aes-128-cbc -e -in f3.txt -out f3_enc.bin -K 00112233445566778889aabbccddeeff -iv 01020304050607080102030405060708
seed@cseed:~/Downloads/Labsetup-arm/Files$ ls -la f1_enc.bin f2_enc.bin f3_enc.bin
-rw-rw-r-- 1 seed seed 16 Feb 21 18:36 f1_enc.bin
-rw-rw-r-- 1 seed seed 32 Feb 21 18:37 f2_enc.bin
-rw-rw-r-- 1 seed seed 32 Feb 21 18:37 f3_enc.bin
seed@cseed:~/Downloads/Labsetup-arm/Files$
```

Figure 21: Encrypting Files

Both f1.txt (5 bytes) and f2.txt (10 bytes) were padded up to 16 bytes so it fits within a single block. However, f3.txt was exactly 16 bytes which required an entire extra padding block to be added, resulting in a final encrypted size of 32 bytes. Each file was decrypted using the -nopad flag, which disables automatic padding removal. The output was then displayed in hex using xxd:

```
openssl enc -aes-128-cbc -d -nopad -in f1_enc.bin -out f1_dec.txt -K
00112233445566778889aabbccddeeff -iv 01020304050607080102030405060708

openssl enc -aes-128-cbc -d -nopad -in f2_enc.bin -out f2_dec.txt -K
00112233445566778889aabbccddeeff -iv 01020304050607080102030405060708

openssl enc -aes-128-cbc -d -nopad -in f3_enc.bin -out f3_dec.txt -K
00112233445566778889aabbccddeeff -iv 01020304050607080102030405060708
```

```
seed@cseed:~/Downloads/Labsetup-arm/Files$ openssl enc -aes-128-cbc -d -nopad -in f1_enc.bin -out f
1_dec.txt -K 00112233445566778889aabbccddeeff -iv 01020304050607080102030405060708
seed@cseed:~/Downloads/Labsetup-arm/Files$ openssl enc -aes-128-cbc -d -nopad -in f2_enc.bin -out f
2_dec.txt -K 00112233445566778889aabbccddeeff -iv 01020304050607080102030405060708
seed@cseed:~/Downloads/Labsetup-arm/Files$ openssl enc -aes-128-cbc -d -nopad -in f3_enc.bin -out f
3_dec.txt -K 00112233445566778889aabbccddeeff -iv 01020304050607080102030405060708
seed@cseed:~/Downloads/Labsetup-arm/Files$
```

Figure 22: Decryption Commands

```
xxd f1_dec.txt
xxd f2_dec.txt
xxd f3_dec.txt
```

```
seed@cseed:~/Downloads/Labsetup-arm/Files$ xxd f1_dec.txt
00000000: 3132 3334 350b 0b0b 0b0b 0b0b 0b0b 0b0b 12345.....
seed@cseed:~/Downloads/Labsetup-arm/Files$
```

Figure 23: f1_dec.txt

```
seed@cseed:~/Downloads/Labsetup-arm/Files$ xxd f2_dec.txt
00000000: 3132 3334 3536 3738 3930 0606 0606 0606 1234567890.....
seed@cseed:~/Downloads/Labsetup-arm/Files$
```

Figure 24: f2_dec.txt

```
seed@cseed:~/Downloads/Labsetup-arm/Files$ xxd f3_dec.txt
00000000: 3132 3334 3536 3738 3930 3132 3334 3536 1234567890123456
00000010: 1010 1010 1010 1010 1010 1010 1010 1010 .....
seed@cseed:~/Downloads/Labsetup-arm/Files$
```

Figure 25: f3_dec.txt

For f1.txt, the plaintext only has 5 bytes leaving 11 bytes needed to reach a full 16 byte block. Eleven padding bytes of value 0x0b (11) are added, bringing the total to 16 bytes. Similarly,

f2.txt has plaintext bytes that require 6 bytes of padding, each with value 0x06 (6) to fill out the remaining space in the block, totalling 16 bytes. In both files, the padding byte value reflects the number of bytes added. For f3.txt, the plaintext already fills an entire 16 byte block, however, padding still requires that an additional full block be added. This means 16 padding bytes, each with value 0x10, are added after the plaintext, resulting in a total encrypted size of 32 bytes. Each padding byte has a value equal to the total number of padding bytes added, allowing the receiver to identify and get rid of the padding after decryption by reading the last byte of the decrypted data.

5. Task 5: Error Propagation – Corrupted Cipher Text

Task 5 involved showing how a single corrupted bit in a ciphertext file impacts the decrypted output under four different AES encryption modes: ECB, CBC, CFB, and OFB.

ECB: Since each block is encrypted and decrypted independently, a corruption in one ciphertext block should only affect the decryption of that single block. The loss should only be 1 block, or 16 bytes of corrupted data.

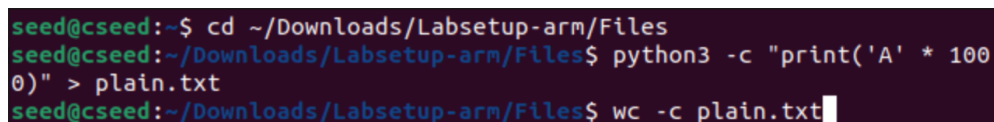
CBC: Each ciphertext block is used as input when decrypting the next block. A corrupted ciphertext block should produce garbage when decrypted and should also corrupt one bit in the next block. The predicted loss should be 1 block, plus 1 bit error in the next block.

CFB: Each ciphertext block also feeds into the decryption of the next. A corruption at byte 54 should corrupt that byte and cause the entire next block to be corrupted. The predicted loss should be 1 byte, plus 1 full block.

OFB: Generates a keystream independently of the ciphertext. A corrupted ciphertext bit should only impact the single next plaintext bit during XOR. The predicted loss should be 1 byte.

A plaintext file of 1001 bytes was created using:

```
python3 -c "print('A' * 1000)" > plain.txt
```



```
seed@cseed:~$ cd ~/Downloads/Labsetup-arm/Files
seed@cseed:~/Downloads/Labsetup-arm/Files$ python3 -c "print('A' * 1000)" > plain.txt
seed@cseed:~/Downloads/Labsetup-arm/Files$ wc -c plain.txt
```

Figure 26: Creating a File

The file was encrypted using AES-128 in all four modes:

```
openssl enc -aes-128-ecb -e -in plain.txt -out long_ecb.bin -K
00112233445566778889aabbccddeeff -nosalt
```

```
openssl enc -aes-128-cbc -e -in plain.txt -out long_cbc.bin -K
00112233445566778889aabbccddeeff -iv 01020304050607080102030405060708 -nosalt

openssl enc -aes-128-cfb -e -in plain.txt -out long_cfb.bin -K
00112233445566778889aabbccddeeff -iv 01020304050607080102030405060708 -nosalt

openssl enc -aes-128-ofb -e -in plain.txt -out long_ofb.bin -K
00112233445566778889aabbccddeeff -iv 01020304050607080102030405060708 -nosalt
```

```
seed@cseed:~/Downloads/Labsetup-arm/Files$ openssl enc -aes-128-ecb -e
-in plain.txt -out long_ecb.bin -K 00112233445566778889aabbccddeeff -
nosalt
seed@cseed:~/Downloads/Labsetup-arm/Files$ openssl enc -aes-128-cbc -e
-in plain.txt -out long_cbc.bin -K 00112233445566778889aabbccddeeff -
iv 01020304050607080102030405060708 -nosalt
seed@cseed:~/Downloads/Labsetup-arm/Files$ openssl enc -aes-128-cfb -e
-in plain.txt -out long_cfb.bin -K 00112233445566778889aabbccddeeff -
iv 01020304050607080102030405060708 -nosalt
seed@cseed:~/Downloads/Labsetup-arm/Files$ openssl enc -aes-128-ofb -e
-in plain.txt -out long_ofb.bin -K 00112233445566778889aabbccddeeff -
iv 01020304050607080102030405060708 -nosalt
seed@cseed:~/Downloads/Labsetup-arm/Files$
```

Figure 27: Encrypting (AES-128) using all modes

The 55th byte (index 54) of each encrypted file was corrupted by flipping one bit using a Python script:

```
data = bytearray(open('long_ecb.bin','rb').read())
data[54] ^= 0x01
open('long_ecb.bin','wb').write(data)
```

```
seed@cseed:~/Downloads/Labsetup-arm/Files$ echo "data = bytearray(open
('long_ecb.bin','rb').read())" > corrupt_ecb.py
seed@cseed:~/Downloads/Labsetup-arm/Files$ echo "data[54] ^= 0x01" >>
corrupt_ecb.py
seed@cseed:~/Downloads/Labsetup-arm/Files$ echo "open('long_ecb.bin','
wb').write(data)" >> corrupt_ecb.py
seed@cseed:~/Downloads/Labsetup-arm/Files$ echo "print('ecb done')" >>
corrupt_ecb.py
seed@cseed:~/Downloads/Labsetup-arm/Files$ python3 corrupt_ecb.py
ecb done
seed@cseed:~/Downloads/Labsetup-arm/Files$
```

Figure 28: Byte Corruption for ECB

```
seed@cseed:~/Downloads/Labsetup-arm/Files$ echo "data = bytearray(open
('long_cbc.bin','rb').read())" > corrupt_cbc.py
seed@cseed:~/Downloads/Labsetup-arm/Files$ echo "data[54] ^= 0x01" >>
corrupt_cbc.py
seed@cseed:~/Downloads/Labsetup-arm/Files$ echo "open('long_cbc.bin','
wb').write(data)" >> corrupt_cbc.py
seed@cseed:~/Downloads/Labsetup-arm/Files$ echo "print('cbc done')" >>
corrupt_cbc.py
seed@cseed:~/Downloads/Labsetup-arm/Files$ python3 corrupt_cbc.py
cbc done
```

Figure 29: Byte Corruption for CBC


```
seed@csseed:~/Downloads/Labsetup-arm/Files$ echo "data = bytearray(open
('long_cfb.bin','rb').read())" > corrupt_cfb.py
seed@csseed:~/Downloads/Labsetup-arm/Files$ echo "data[54] ^= 0x01" >>
corrupt_cfb.py
seed@csseed:~/Downloads/Labsetup-arm/Files$ echo "open('long_cfb.bin','
wb').write(data)" >> corrupt_cfb.py
seed@csseed:~/Downloads/Labsetup-arm/Files$ echo "print('cfb done')" >>
corrupt_cfb.py
seed@csseed:~/Downloads/Labsetup-arm/Files$ python3 corrupt_cfb.py
cfb done
seed@csseed:~/Downloads/Labsetup-arm/Files$
```

Figure 30: Byte Corruption for CFB

```
seed@csseed:~/Downloads/Labsetup-arm/Files$ echo "data = bytearray(open
('long_ofb.bin','rb').read())" > corrupt_ofb.py
seed@csseed:~/Downloads/Labsetup-arm/Files$ echo "data[54] ^= 0x01" >>
corrupt_ofb.py
seed@csseed:~/Downloads/Labsetup-arm/Files$ echo "open('long_ofb.bin','
wb').write(data)" >> corrupt_ofb.py
seed@csseed:~/Downloads/Labsetup-arm/Files$ echo "print('ofb done')" >>
corrupt_ofb.py
seed@csseed:~/Downloads/Labsetup-arm/Files$ python3 corrupt_ofb.py
ofb done
```

Figure 31: Byte Corruption for OFB

The same script was repeated for each mode. Then each corrupted file was decrypted using the correct key and IV with the -nopad flag to prevent automatic padding removal.

```
seed@csseed:~/Downloads/Labsetup-arm/Files$ openssl enc -aes-128-ecb -d
-in long_ecb.bin -out dec_ecb.txt -K 00112233445566778889aabbccddeeff
-nosalt -nopad
seed@csseed:~/Downloads/Labsetup-arm/Files$ openssl enc -aes-128-cbc -d
-in long_cbc.bin -out dec_cbc.txt -K 00112233445566778889aabbccddeeff
-iv 01020304050607080102030405060708 -nosalt -nopad
seed@csseed:~/Downloads/Labsetup-arm/Files$ openssl enc -aes-128-cfb -d
-in long_cfb.bin -out dec_cfb.txt -K 00112233445566778889aabbccddeeff
-iv 01020304050607080102030405060708 -nosalt -nopad
seed@csseed:~/Downloads/Labsetup-arm/Files$ openssl enc -aes-128-ofb -d
-in long_ofb.bin -out dec_ofb.txt -K 00112233445566778889aabbccddeeff
-iv 01020304050607080102030405060708 -nosalt -nopad
seed@csseed:~/Downloads/Labsetup-arm/Files$
```

Figure 32: Decryption

The results matched the predictions, ECB contained corruption to exactly one 16 byte block, with all other blocks recovering. CBC and CFB both extended errors to adjacent blocks, in CBC, the corrupted ciphertext block produced corruption after decryption and also flipped one bit in the next plaintext block. CFB showed a single corrupted byte that appeared first, followed by a fully corrupted next block. OFB was the most resilient, with only a single byte corrupted at the position of the flipped bit.

```
seed@csseed:~/Downloads/Labsetup-arm/Files$ xxd dec_ecb.txt | sed -n '3
,7p'
00000020: 4141 4141 4141 4141 4141 4141 4141 4141  AAAAAAAAAAAAAAAAAA
00000030: 446d 73b9 06e4 c77a 158c 94b8 8b83 4306  Dms....Z.....C.
00000040: 4141 4141 4141 4141 4141 4141 4141 4141  AAAAAAAAAAAAAAAAAA
00000050: 4141 4141 4141 4141 4141 4141 4141 4141  AAAAAAAAAAAAAAAAAA
00000060: 4141 4141 4141 4141 4141 4141 4141 4141  AAAAAAAAAAAAAAAAAA
```

Figure 33: ECB Corruption

```
seed@cseed:~/Downloads/Labsetup-arm/Files$ xxd dec_cbc.txt | sed -n '3
,7p'
00000020: 4141 4141 4141 4141 4141 4141 4141 4141  AAAAAAAAAAAAAAAAAA
00000030: 5834 3d4b 7906 17e8 ddf6 c3c4 1760 c397  X4=Ky.....`..
00000040: 4141 4141 4141 4041 4141 4141 4141 4141  AAAAAA@AAAAAAAAA
00000050: 4141 4141 4141 4141 4141 4141 4141 4141  AAAAAAAAAAAAAAAAAA
00000060: 4141 4141 4141 4141 4141 4141 4141 4141  AAAAAAAAAAAAAAAAAA
```

Figure 33: CBC Corruption

```
seed@cseed:~/Downloads/Labsetup-arm/Files$ xxd dec_cfb.txt | sed -n '3
,7p'
00000020: 4141 4141 4141 4141 4141 4141 4141 4141  AAAAAAAAAAAAAAAAAA
00000030: 4141 4141 4141 4041 4141 4141 4141 4141  AAAAAA@AAAAAAAAA
00000040: 1d33 2ff1 8e3c 1846 641a 3d0c 379e 8a4c  .3/..<.Fd.=.7..L
00000050: 4141 4141 4141 4141 4141 4141 4141 4141  AAAAAAAAAAAAAAAAAA
00000060: 4141 4141 4141 4141 4141 4141 4141 4141  AAAAAAAAAAAAAAAAAA
```

Figure 34: CFB Corruption

```
seed@cseed:~/Downloads/Labsetup-arm/Files$ xxd dec_ofb.txt | sed -n '3
,7p'
00000020: 4141 4141 4141 4141 4141 4141 4141 4141  AAAAAAAAAAAAAAAAAA
00000030: 4141 4141 4141 4041 4141 4141 4141 4141  AAAAAA@AAAAAAAAA
00000040: 4141 4141 4141 4141 4141 4141 4141 4141  AAAAAAAAAAAAAAAAAA
00000050: 4141 4141 4141 4141 4141 4141 4141 4141  AAAAAAAAAAAAAAAAAA
00000060: 4141 4141 4141 4141 4141 4141 4141 4141  AAAAAAAAAAAAAAAAAA
```

Figure 35: OFB Corruption

6. Task 6: Initial Vector (IV) and Common Mistakes

Task 6 involves the importance of properly selecting Initial Vectors in encryption and shows what happens when IVs are reused or are predictable.

Task 6.1. IV Experiment

A test message was created and encrypted three times using AES-128-CBC:

```
echo -n "This is a test message" > msg.txt
```

Encryption #1:

```
openssl enc -aes-128-cbc -e -in msg.txt -out iv_test1.bin -K
00112233445566778889aabbccddeeff -iv 01020304050607080102030405060708 -p
```

```
seed@cseed:~/Downloads/Labsetup-arm/Files$ openssl enc -aes-128-cbc -e
-in msg.txt -out iv_test1.bin -K 00112233445566778889aabbccddeeff -iv
01020304050607080102030405060708 -p
salt=002081DBAAAA0000
key=00112233445566778889AABBCCDDEEFF
iv =01020304050607080102030405060708
seed@cseed:~/Downloads/Labsetup-arm/Files$
```

Figure 36: Encryption #1

Encryption #2:

```
openssl enc -aes-128-cbc -e -in msg.txt -out iv_test2.bin -K
00112233445566778889aabbccddeeff -iv 00000000000000000000000000000000 -p
```

```
seed@cseed:~/Downloads/Labsetup-arm/Files$ openssl enc -aes-128-cbc -e
-in msg.txt -out iv_test2.bin -K 00112233445566778889aabbccddeeff -iv
00000000000000000000000000000000 -p
salt=0020B4ABAAAA0000
key=00112233445566778889AABBCCDDEEFF
iv =00000000000000000000000000000000
seed@cseed:~/Downloads/Labsetup-arm/Files$
```

Figure 37: Encryption #2

Encryption #3:

```
openssl enc -aes-128-cbc -e -in msg.txt -out iv_test3.bin -K
00112233445566778889aabbccddeeff -iv 01020304050607080102030405060708 -p
```

```
seed@cseed:~/Downloads/Labsetup-arm/Files$ openssl enc -aes-128-cbc -e
-in msg.txt -out iv_test3.bin -K 00112233445566778889aabbccddeeff -iv
01020304050607080102030405060708 -p
salt=002020EAAAAA0000
key=00112233445566778889AABBCCDDEEFF
iv =01020304050607080102030405060708
seed@cseed:~/Downloads/Labsetup-arm/Files$
```

Figure 38: Encryption #3

The three ciphertext files were then compared using xxd:

```
xxd iv_test1.bin
xxd iv_test2.bin
xxd iv_test3.bin
```

```
seed@cseed:~/Downloads/Labsetup-arm/Files$ xxd iv_test1.bin
00000000: 1e09 f102 5667 63ee b2f3 cdcc 833b 4eae ....Vgc.....;N.
00000010: fa69 a8e3 2c8b f25b fc37 a57c 35e2 35b7 .i.,...[.7.]5.5.
seed@cseed:~/Downloads/Labsetup-arm/Files$ xxd iv_test2.bin
00000000: 51a3 f62f c2fc 5a1b 5c83 9bef 0543 e7a8 Q../..Z.\....C..
00000010: deaf dfa4 c29a fcac 4569 c4ac 9d71 9dd7 .....Ei...q..
seed@cseed:~/Downloads/Labsetup-arm/Files$ xxd iv_test3.bin
00000000: 1e09 f102 5667 63ee b2f3 cdcc 833b 4eae ....Vgc.....;N.
00000010: fa69 a8e3 2c8b f25b fc37 a57c 35e2 35b7 .i.,...[.7.]5.5.
seed@cseed:~/Downloads/Labsetup-arm/Files$
```

Figure 39: All 3 Test outputs

The output shows that 'iv_test1.bin' and 'iv_test3.bin' are identical, while 'iv_test2.bin' is different despite encrypting the exact same plaintext with the same key with the only difference being the IV used. When the same plaintext is encrypted with the same key and IV, the ciphertext is always identical, so the attacker can detect when the same message has been sent

twice without ever breaking the encryption. Using a different IV outputs a different ciphertext that reveals nothing about the relationship between the two messages.

Task 6.2. Common Mistake: Use the Same IV

In OFB mode, encryption works:

$$C = P \text{ (XOR) } \text{KeyStream}$$

Since the keystream depends only on the key and IV, if the same IV is reused the keystream is identical for both messages:

$$\text{KeyStream} = P1 \text{ (XOR) } C1$$

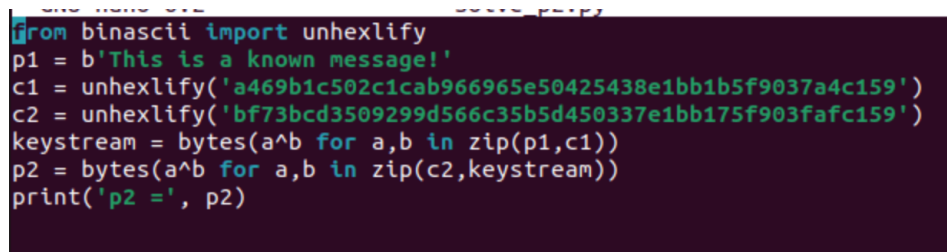
$$P2 = C2 \text{ (XOR) } \text{KeyStream} = C2 \text{ (XOR) } C1 \text{ (XOR) } P1$$

Given Information:

- P1 = This is a known message!
- C1 = a469b1c502c1cab966965e50425438e1bb1b5f9037a4c159
- C2 = bf73bcd3509299d566c35b5d450337e1bb175f903fafc159

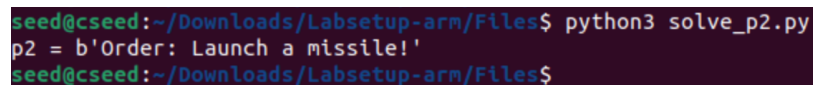
Solution Script:

```
from binascii import unhexlify
p1 = b'This is a known message!'
c1 = unhexlify('a469b1c502c1cab966965e50425438e1bb1b5f9037a4c159')
c2 = unhexlify('bf73bcd3509299d566c35b5d450337e1bb175f903fafc159')
keystream = bytes(a^b for a,b in zip(p1,c1))
p2 = bytes(a^b for a,b in zip(c2,keystream))
print('P2 =', p2)
```



```
from binascii import unhexlify
p1 = b'This is a known message!'
c1 = unhexlify('a469b1c502c1cab966965e50425438e1bb1b5f9037a4c159')
c2 = unhexlify('bf73bcd3509299d566c35b5d450337e1bb175f903fafc159')
keystream = bytes(a^b for a,b in zip(p1,c1))
p2 = bytes(a^b for a,b in zip(c2,keystream))
print('P2 =', p2)
```

Figure 40: Python Script



```
seed@cseed:~/Downloads/Labsetup-arm/Files$ python3 solve_p2.py
p2 = b'Order: Launch a missile!'
seed@cseed:~/Downloads/Labsetup-arm/Files$
```

Figure 41: Output

If OFB were replaced with CFB mode, only the first block of P2 could be revealed using this attack, since in CFB mode the keystream for each block depends on the previous ciphertext block rather than just the IV, meaning an attacker with only P1 and C1 could derive the keystream for the first block and recover the first 16 bytes of P2, but would lack the ciphertext blocks needed to decrypt anything past that.

Task 6.3. Common Mistake: Use a Predictable IV

In CBC mode, encryption of the first block is computed as:

$$C = \text{AES_K}(P \text{ XOR } IV)$$

An implementation is secure if both IVs are unique and unpredictable. If an attacker can predict the next IV, they can mount a chosen-plaintext attack to determine the content of an encrypted message even without knowing the key.

After connecting to the oracle, the following values were obtained:

- Bob's ciphertext: 6f6988945c4fb082a7e7e48ad858fa66
- The IV used: f38b6738cdc71c046c1ed3991923967f
- Next IV: 46247339cdc71c046c1ed3991923967f

The following Python script was written to compute the crafted plaintext. The Python script was written to automate the computation of the crafted attack plaintext:

```
def xor(a, b):
    return bytes(x ^ y for x, y in zip(a, b))

C_bob = bytes.fromhex("6f6988945c4fb082a7e7e48ad858fa66")
IV_bob = bytes.fromhex("f38b6738cdc71c046c1ed3991923967f")
IV_next = bytes.fromhex("46247339cdc71c046c1ed3991923967f")

P_yes = b"Yes" + bytes([13] * 13)
P_no = b"No" + bytes([14] * 14)

P_attack_yes = xor(xor(P_yes, IV_bob), IV_next)
P_attack_no = xor(xor(P_no, IV_bob), IV_next)

print("Guess YES ->", P_attack_yes.hex())
print("Guess NO ->", P_attack_no.hex())
```

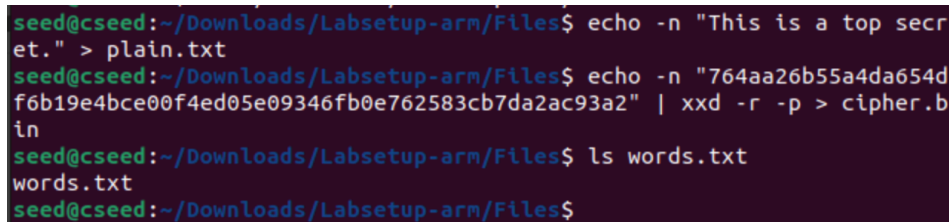

Comparing the first 16 bytes of the oracle's returned ciphertext against 'C_bob', both values are identical. Since the first block of the oracle's output matches 'C_bob', this confirms that the guess was correct and that Bob's secret message is 'Yes'. This attack succeeds because by knowing 'IV_next' in advance, the attacker is able to craft a plaintext that neutralized the IV's effect, causing the oracle to encrypt the guessed message under Bob's original IV.

7. Task 7: Programming using the Crypto Library

Task 7 involved writing a C program to find an unknown AES-128-CBC encryption key through brute force search. It iterates through every word in a dictionary file, builds the candidate key, encrypts the known plaintext, and compares the result against known ciphertext.

Given Information

- Plaintext: This is a top secret.
- Ciphertext: 764aa26b55a4da654df6b19e4bce00f4ed05e09346fb0e762583cb7da2ac93a2
- IV: aabbccddeeff00998877665544332211



```
seed@cseed:~/Downloads/Labsetup-arm/Files$ echo -n "This is a top secret." > plain.txt
seed@cseed:~/Downloads/Labsetup-arm/Files$ echo -n "764aa26b55a4da654df6b19e4bce00f4ed05e09346fb0e762583cb7da2ac93a2" | xxd -r -p > cipher.bin
seed@cseed:~/Downloads/Labsetup-arm/Files$ ls words.txt
words.txt
seed@cseed:~/Downloads/Labsetup-arm/Files$
```

Figure 47: Given Information

The OpenSSL development library was installed to provide access to the EVP encryption functions:

```
sudo apt-get install libssl-dev
```

Program Design:

1. Open the dictionary file words.txt
2. For each word shorter than 16 characters, construct a 16 byte key by copying the word into a buffer filled with (#) characters
3. Encrypt the known plaintext using AES-128-CBC with the candidate key and the known IV
4. Compare the first 32 bytes of the output against the known ciphertext
5. If they match, the key has been found

```

#include <stdio.h>
#include <string.h>
#include <openssl/evp.h>

int main() {
    FILE *wordfile = fopen("words.txt", "r");
    char word[100];
    unsigned char key[16];
    unsigned char iv[] = {0xaa,0xbb,0xcc,0xdd,0xee,0xff,0x00,0x99,0x88,0x77,0x66,0x55,0x44,0x33,0x22,0x11};
    unsigned char plaintext[] = "This is a top secret.";
    unsigned char ciphertext[] = {0x76,0x4a,0xa2,0x6b,0x55,0xa4,0xda,0x65,0x4d,0xf6,0xb1,0x9e,0x4b,0xce,0x00,0xf4,0xed,0x00,0x00,0x00,0x00};
    unsigned char outbuf[64];
    int outlen;
    int tmplen;
    while (fgets(word, sizeof(word), wordfile)) {
        word[strcspn(word, "\n")] = 0;
        if (strlen(word) >= 16) {
            memset(key, word, strlen(word));
            EVP_CIPHER_CTX *ctx = EVP_CIPHER_CTX_new();
            EVP_EncryptInit_ex(ctx, EVP_aes_128_cbc(), NULL, key, iv);
            EVP_EncryptUpdate(ctx, outbuf, &outlen, plaintext, strlen((char*)plaintext));
            EVP_EncryptFinal_ex(ctx, outbuf + outlen, &tmplen);
            EVP_CIPHER_CTX_free(ctx);
            if (memcmp(outbuf, ciphertext, 32) == 0) {
                printf("Key: %s\n", word);
                fclose(wordfile);
                return 0;
            }
        }
    }
    printf("Not found");
    fclose(wordfile);
    return 0;
}

```

Figure 48: Python script

The program was compiled using gcc with the -lcrypto flag to link the OpenSSL crypto library:

```
gcc -o brute brute.c -lcrypto
./brute
```

```

seed@cseed:~/Downloads/Labsetup-arm/Files$ gcc -o brute brute.c -lcrypto
brute.c: In function 'main':
brute.c:18:13: warning: passing argument 2 of 'memset' makes integer from pointer without a cast [-Wint-conversion]
   18 | memset(key, word, strlen(word));
      |             ^~~~~
      |             |
      |             char *
In file included from brute.c:2:
/usr/include/string.h:61:37: note: expected 'int' but argument is of type 'char *'
   61 | extern void *memset (void *__s, int __c, size_t __n) __THROW __nonnull (
      |                               ^~~~~
seed@cseed:~/Downloads/Labsetup-arm/Files$ ./brute
Syracuse@cseed:~/Downloads/Labsetup-arm/Files$

```

Figure 49: Compilation and output

The program searched through the dictionary and identified the encryption key, which was padded with (#) characters to form the full 16 byte AES key used for encryption. The 'EVP_CIPHER_CTX' interface was chosen over lower level OpenSSL functions for its cleaner API and automatic padding handling, and the 'memcmp' comparison limited to 32 bytes to confirm matches easier. Words of 16 characters or more are skipped, since they can't fit into the key buffer without overwriting the padding.

References:

Du, W. (2019). *Computer & internet security: A hands-on approach* (2nd ed.). Wenliang Du. <https://www.handsonsecurity.net>

National Institute of Standards and Technology. (2001). *Recommendation for block cipher modes of operation* (NIST Special Publication 800-38A). U.S. Department of Commerce. <https://nvlpubs.nist.gov/nistpubs/legacy/sp/nistspecialpublication800-38a.pdf>

OpenSSL Foundation. (2021). *EVP symmetric encryption and decryption*. OpenSSL Documentation. https://www.openssl.org/docs/man1.1.1/man3/EVP_CipherInit.html

OpenSSL Wiki. (n.d.). *Enc*. OpenSSL Community Wiki. <https://wiki.openssl.org/index.php/Enc>

Wikipedia. (2025, February). *Frequency analysis*. Wikimedia Foundation. https://en.wikipedia.org/wiki/Frequency_analysis

Wikipedia. (2024, January). *Block cipher mode of operation*. Wikimedia Foundation. https://en.wikipedia.org/wiki/Block_cipher_mode_of_operation

Python Software Foundation. (2024). *Python on Windows FAQ*. Python 3 Documentation. <https://docs.python.org/3/faq/windows.html>

Tutorialspoint. (n.d.). *Cryptography with Python — XOR process*. https://www.tutorialspoint.com/cryptography_with_python/cryptography_with_python_xor_process.htm

Python Software Foundation. (2024). *binascii — Convert between binary and ASCII*. Python 3 Documentation. <https://docs.python.org/3/library/binascii.html>